

UC Berkeley · Master of Financial Engineering



MFE Python Winter 2026

Lesson 1

Course intro, Git, Python, and AI

October 5, 2026

Welcome!

Ask questions as we go.

Today: get set up, write Python, and use AI.

Today's Agenda

Course introduction	15 min
Git fundamentals — version control & workflows	40 min
<i>Break</i>	10 min
Python environment — uv & virtual environments	40 min
Basic Python	40 min
<i>Break</i>	5 min
AI tools in programming	15 min
Wrap-up & next steps	10 min

Meet Your Instructors

Yang Guo

CIO, Opendoor

Former leadership roles at Alt, Citi & Uber

Double Berkeley alum
MFE; BS in Economics & Molecular Biology

Expertise: portfolio risk management and growth strategy

Aneesh Sachdeva

Founder, Glia Intelligence

Former ML leadership roles at Walmart, Alt & Uber

Expertise: labor transformation, LLM deployments and supply chain forecasting

Course Overview

 **Python Proficiency**

Data Analysis & Visualization 

Designed for Financial Engineers &
Quantitative Analysts

 **Machine Learning Foundations**

Production Practices 

What You'll Learn

1.  **Python Programming**
 - Git, Package Management, Data Structs, Advanced Practices
2.  **Data Analysis & Visualization**
 - Pandas, Matplotlib, SQL
3.  **Machine Learning**
 - Sklearn Fundamentals
4.  **Production Code**
 - Testing, Documentation, Best Practices

Course Structure

Format

- 10 Sessions (3 hours each)
- Virtual Zoom lectures
- Recorded for async viewing
- Active Q&A encouraged

Tools

- Canvas (Course Materials)
- Slack (Communications)
- GitHub (Assignments)
- Python Environment (Local Development)

Course Schedule • Oct 5–Nov 5, 2026

All sessions: 9 a.m.–noon Pacific • 12–3 p.m. Eastern

Date	Instructor	Topic / focus
Mon 10/5	Aneesh	Basics • Python syntax, Git, CLI, AI tools
Thu 10/8	Yang	Data Analysis I & II • pandas, plotting
Mon 10/12	Aneesh	LLM Classification & Agent Evaluation
Thu 10/15	Yang	Data Analysis III • SQL
Mon 10/19	Aneesh	Advanced Python I • OOP, environments, regex
Thu 10/22	Aneesh	Advanced Python II • async, functional programming
Mon 10/26	Yang	Modeling I • scikit-learn basics
Thu 10/29	Yang	Modeling II • scikit-learn advanced
Mon 11/2	Aneesh	Productionization • testing, Docker, CI
Thu 11/5	Yang	Advanced topic • on demand

Grading & Assignments

Grading System

- Pass/No Pass
- Based on assignment completion

Assignments

- 4-5 Coding Projects
- Individual Work
- GitHub Submission
- Final project • details to follow

Grading & Assignments

Late Policy

- Each student will be given three free late days, with each late day being equivalent to 24 hours of lateness.
- Assignments are considered either on time or late by whole days; for instance, an assignment is either on time, 1 day late, 2 days late, etc.
- Due to the leniency of this policy, no extensions will be granted for assignments under any circumstances.
- No assignment will be accepted if it is submitted more than 3 days after its due date.

Success Criteria

- Complete assignments on time
- Follow coding standards
- Show understanding of concepts

How to Succeed

Course Goals

- Tech/Quant Interview Ready
- Industry-Standard Practices
- Independent Development Skills

Success Strategies

- Watch recordings if you miss live sessions
- **Start assignments early**
- Engage in Slack discussions
- Collaborate with AI when stuck

Common Pitfalls to Avoid

- Waiting until deadlines
- Skipping environment setup
- Not asking for help when stuck

Getting Help

Course Communication

- Slack: Real-time discussions & questions
- Canvas: Announcements & materials
- Email: Administrative matters

Support Resources

- Live Q&A during lectures
- Slack channel for tech support
- Assignment feedback
- Reference materials & guides
- ChatGPT / Claude AI

Why AI is central to this course

How we got here

Homework was intentional

Each year, we raised the difficulty to push you to use AI—and learn how to communicate with it.

We hesitated to teach AI more deeply while workplace adoption and interviews lagged.

Why we're leaning in

Prepare for interviews and work

We now see AI adoption even in legacy companies.

Our expectation: Python knowledge and AI fluency together improve your readiness for the roles you want.

Learn Python. Learn to direct AI.

Python fluency

Understand from first principles

Write and explain code yourself.

Reason about data structures, logic, performance, and tradeoffs.

Be ready to solve interview problems independently.

AI fluency

Turn intent into useful work

Give clear goals, context, and constraints.

Read, test, and improve the code AI writes.

Direct AI in data science and quantitative analysis.

Use AI as your personal tutor

Learn actively

Explanation → practice → feedback

Ask for examples, hints, and questions.

Try the code yourself and explain it back.

Use feedback, then solve a new problem without the answer in front of you.

Our teaching approach

Concepts, then guided practice

We will spend less time on Python's syntax minutiae.

We will introduce concepts and teach you a path for self-study.

Use AI to make that practice more effective.

Understand the system behind the code

From bits to atoms

Python runs on physical systems

CPU: executes instructions and coordinates work.

Memory: holds data and program state.

GPU: accelerates suitable parallel workloads.

Think in resources

One server → a cluster of servers

How is work coordinated across machines?

What limits throughput: compute, memory, or data movement?

What are the tradeoffs in latency, cost, and scale?

What you should leave able to do

Your capabilities

Speak the language of an engineer

Discuss Python with an advanced engineer.

Understand and explain AI-generated code.

Direct AI to perform data science and quantitative analysis.

Your role as an engineer

Solve problems; use judgment

AI changes how we do the work.

Engineering still means understanding the problem and building a solution.

Keep adapting where you focus your learning.

Ready to Begin!

Let's start your python journey!

Up Next: Git Fundamentals

Remember: Ask questions anytime via chat or voice

Introduction to Git

What is Version Control?

Without Version Control:

- Multiple file copies
- Lost changes
- Confused which version is latest
- Hard to collaborate

report_final.py

Oct 1

report_final_v2.py

Oct 3

report_final_LATEST.py

Oct 5

report_final_ACTUALLY_FINAL.py

Oct 6

Enter Git

Git is:

- A version control system
- Like a time machine for your code
- Industry standard tool
- Free and open source

Created by:

Linus Torvalds (2005)

The same person who created Linux!

Why Git Matters



Collaboration

- Multiple people can work together
- Track who changed what
- Merge changes safely



Code Review

- Review others' code
- Comment and suggest
- Maintain quality



Version Control

- Track all changes
- Restore previous versions
- Branch for experiments



Backup

- Safe storage
- Multiple copies
- Disaster recovery

Git Basic Concepts

Repository (Repo)

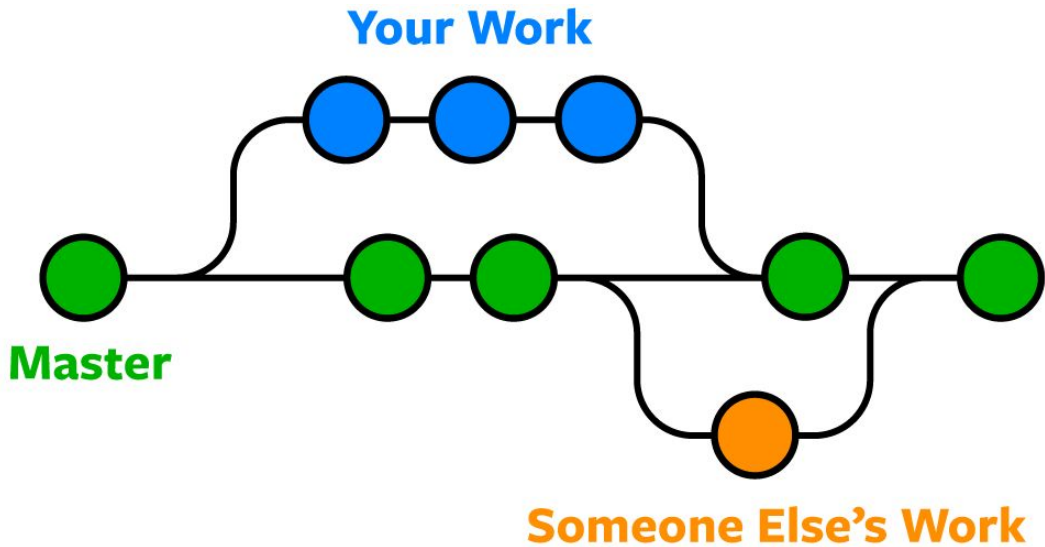
- A project's folder with version tracking
- Contains all project files and history

Commit

- A snapshot of your changes
- Like a checkpoint in a game

Branch

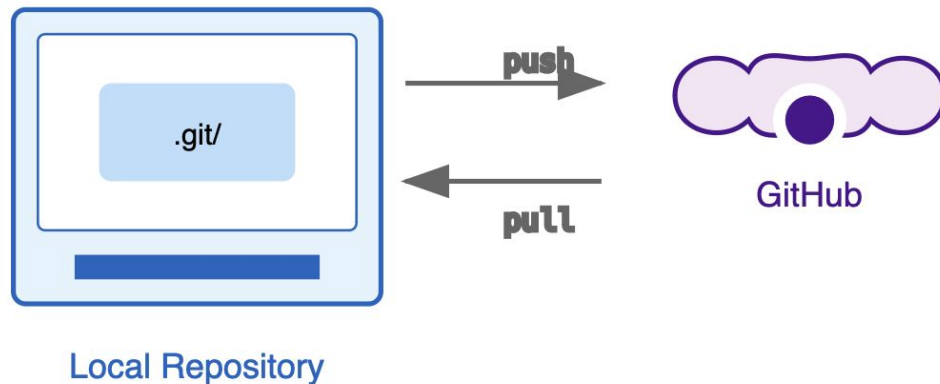
- Separate line of development
- Like a parallel universe for your code



Local vs. Remote

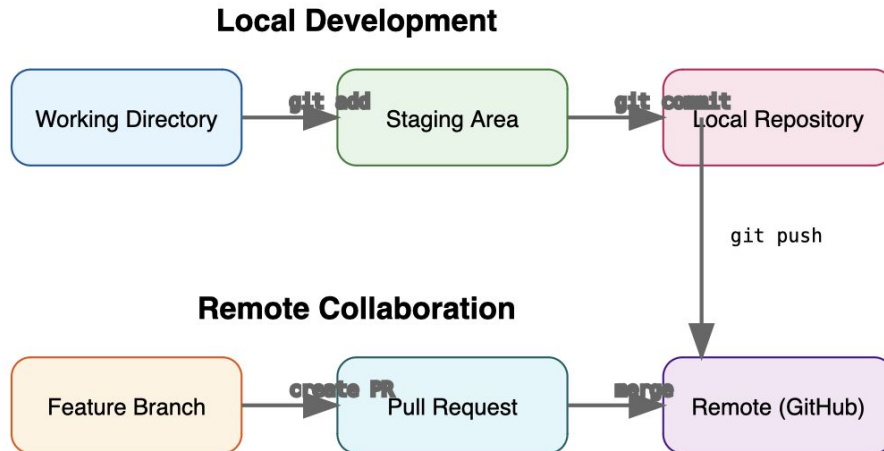
Local Repository

- On your computer
- Where you work
- Full project history



Remote Repository

- On the internet (e.g., GitHub)
- Shared with team
- Backup of your work



Getting Started

1. Install Git

- Windows: <https://git-scm.com/download/win>
- Mac: Use `brew install git`
- Linux: `sudo apt-get install git`

2. Configure Git

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your.email@example.com"
```

3. Verify Installation

```
git --version
```

Basic Git Commands

Repository setup

<code>git init</code>	<code># create new repo</code>
<code>git clone <repository-url></code>	<code># copy existing repo</code>

Core Commands

<code>git status</code>	<code># check file status</code>
<code>git add filename</code>	<code># stage specific file</code>
<code>git add .</code>	<code># stage all changes</code>
<code>git commit -m "Your message here"</code>	<code># save changes</code>

Branch Management

<code>git checkout -b hw_1_firstname_lastname</code>	<code># create & switch branch</code>
<code>git checkout main</code>	<code># switch to main branch</code>
<code>git branch</code>	<code># list branches</code>

Complete Git Workflow (local)

Do your work locally first:

1. Clone repository
2. Create feature branch
3. Make changes
4. Stage changes (git add)
5. Commit changes (git commit)

Complete Git Workflow (remote)

Now push your work to the remote repo (i.e. share):

1. Push to Github

First time push for new branch

```
git push --set-upstream origin hw_1_firstname_lastname
```

Subsequent pushes

```
git push
```

1. Create Pull Request (PR):
 - Go to GitHub repository ([course repository link](#))
 - Click "New Pull Request"
 - Select your branch
 - Assign PRs to kevinramlal and elfiasco
 - Submit PR

Git Demo

Best Practices

✓ Do:

- Commit often
- Write clear commit messages (≤ 50 characters)
- Pull before starting work (`git pull`)
- Use branches for features

✗ Don't:

- Commit broken code
- Share sensitive data
- Commit large files
- Ignore merge conflicts

Common Conflicts

Merge Conflicts

- What: Two people changed same code
- Solution: Manual resolution required

Large Files

- What: Git slows with big files
- Solution: Use Git LFS or don't track

Authentication

- What: Permission denied
- Solution: Setup SSH keys properly

Merge Conflicts

What is a Merge Conflict?

- Occurs when Git can't automatically merge changes
- Multiple people modified the same lines
- Git needs human help to decide what to keep

How to Resolve:

1. Find files with conflicts (marked with **✖** in `git status`)
2. Open files and look for conflict markers
3. Choose which changes to keep:
 - Keep your changes (HEAD)
 - Keep their changes (feature_branch)
 - Keep both
 - Write something completely new
4. Remove conflict markers
5. Save file
6. `git add` the resolved files
7. `git commit` to complete the merge

```
<<<<<< HEAD
print("Hello from main branch!")
=====
print("Hello from feature branch!")
>>>>>> feature_branch
```

Best Practices:

- Pull main branch often
- Communicate with team
- Break work into smaller pieces

Essential Bash Commands

Navigation & Files:

```
cd path/to/dir      # Change directory
cd ..               # Go up one directory
cd ~                # Go to home directory
ls                  # List files
ls -la              # List all files with details
pwd                 # Print working directory
```

File Operations:

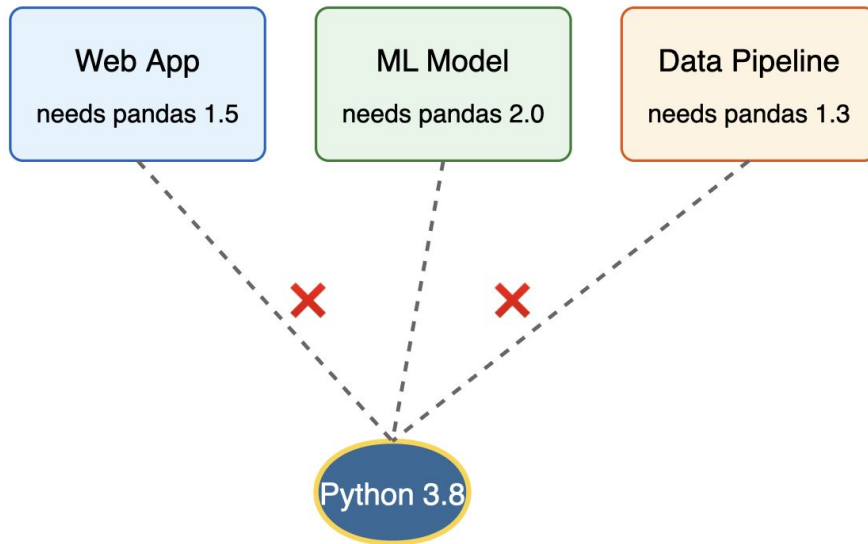
```
touch file.txt      # Create empty file
cat file.txt         # Display file contents
cp file1 file2       # Copy file1 to file2
mv old.txt new.txt   # Move/rename file
rm file.txt          # Delete file
rm -r dir            # Delete directory
mkdir dirname        # Create directory
echo "text" > f.txt  # Write text to file
echo "more" >> f.txt # Append text to file
```

Python Environment Setup

The Python Environment Problem

Without Virtual Environments:

- One Python installation for everything
- Package conflicts between projects
- "Works on my machine" syndrome
- Difficult version management



Enter uv

What is uv?

- Package and environment manager
- Cross-platform (Windows, Mac, Linux)
- Extremely fast (written in Rust)
- Drop-in replacement for pip and venv

Why uv?

- Isolation between projects
- Reproducible environments
- Easy package management

Installing uv

Install uv

- Mac/Linux: `curl -LsSf https://astral.sh/uv/install.sh | sh`
- Windows: `powershell -c "irm https://astral.sh/uv/install.ps1 | iex"`

Or via Homebrew (Mac)

- `brew install uv`
- Restart terminal after install
- See `docs.astral.sh/uv` for more options

Verify Installation

- `uv --version`
- `uv python list`

Core uv Commands

Environment Management

Command	Description
<code>uv venv</code>	Create virtual environment
<code>source .venv/bin/activate</code>	Activate environment (Mac/Linux)
<code>.venv\Scripts\activate</code>	Activate environment (Windows)
<code>deactivate</code>	Deactivate environment
<code>uv python install 3.12</code>	Install Python version

Package Management

Command	Description
<code>uv pip install pandas</code>	Install a package
<code>uv pip install -r req.txt</code>	Install from requirements
<code>uv pip list</code>	List installed packages
<code>uv pip uninstall pandas</code>	Uninstall a package

Reproducible Course Setup

pyproject.toml + uv.lock

pyproject.toml	→ declared dependencies
uv.lock	→ resolved versions
.venv/	→ local Python environment

Run commands from the course repo root.

Install the course's locked dependencies

```
uv sync --locked
```

Select .venv as your notebook kernel

```
uv run python hello.py
```

Best Practices

✓ Do:

- One environment per project
- Use pyproject.toml + uv.lock
- Track requirements in version control
- Keep environments minimal

✗ Don't:

- Mix package managers (stick to uv)
- Work without a virtual environment
- Forget to activate environment
- Install unnecessary packages

Common Issues

Package Conflicts

- Problem: Incompatible versions
- Solution: Check dependency documentation

Installation Failures

- Problem: Build errors
- Solution: uv uses pre-built wheels by default

Path Issues

- Problem: Wrong Python version
- Solution: Always check **which python**

uv Demo

Basic Python Programming

Why Python?

Advantages for Finance & Data Science:

- Simple, readable syntax
- Rich ecosystem of libraries
- Strong data processing capabilities
- Active community support

Industry Adoption:

- Major banks and financial institutions
- Tech companies
- Research institutions
- Quant trading firms
- Widely used for AI/ML

Python Basics: Running Code

Interactive Shell:

```
>>> print("Hello, World!")  
Hello, World!  
>>> 2 + 2  
4
```

Script Files (.py)

```
# hello.py  
print("Hello, World!")
```

^run with: `python hello.py`

Variables and Data Types

Type	Example	Description
<code>int</code>	<code>x = 42</code>	Integer numbers
<code>float</code>	<code>y = 3.14</code>	Decimal numbers
<code>str</code>	<code>s = "Hello"</code>	Text
<code>bool</code>	<code>b = True</code>	True/False
<code>None</code>	<code>n = None</code>	Null value

```
# Type checking
type(42)      # <class 'int'>
type("Hello") # <class 'str'>
```

Lists: ordered, mutable sequences

List Creation & Access

```
# Creating lists
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]

# Accessing elements
print(numbers[0])      # 1 (first element)
print(numbers[-1])     # 5 (last element)
print(numbers[1:3])    # [2, 3] (slicing)
```

List Operations:

```
# Modifying lists
numbers.append(6)      # Add to end
numbers.extend([7,8])  # Add multiple items
numbers.pop()          # Remove last item
numbers.insert(0, 0)   # Insert at position
```

Strings

String Creation & Methods:

```
# Multiple ways to create strings
```

```
s1 = 'Single quotes'
```

```
s2 = "Double quotes"
```

```
s3 = """Multi-line  
string"""
```

```
# Common operations
```

```
name = "Python"
```

```
print(len(name))           # 6
```

```
print(name.upper())        # PYTHON
```

```
print(name.lower())        # python
```

```
print(name[0])             # P
```

```
print(name[1:4])           # yth
```

String Formatting:

```
name = "Alice"
```

```
age = 25
```

```
f"My name is {name} and I'm {age}"    # f-strings
```

```
"My name is {} and I'm {}".format(name, age)
```

***Immutable
sequences***

Numbers and Math

Basic Operations:

```
# Arithmetic
print(10 + 5)    # 15  (addition)
print(10 - 5)    # 5   (subtraction)
print(10 * 5)    # 50  (multiplication)
print(10 / 5)    # 2.0 (division)
print(10 // 3)   # 3   (floor division)
print(10 % 3)    # 1   (modulo)
print(2 ** 3)    # 8   (exponentiation)
```

Type Conversion:

```
# Casting between types
int(3.14)        # 3
float(3)         # 3.0
str(123)         # "123"
```

Dictionaries

Dictionary Basics:

```
# Creating dictionaries
person = {
    "name": "Alice",
    "age": 25,
    "city": "SF"
}

# Accessing values
print(person["name"])    # Alice
print(person.get("age")) # 25
```

Dictionary Operations:

```
# Modifying dictionaries
person["job"] = "Engineer" # Add new key-value
person.update({"age": 26}) # Update existing
keys = person.keys()       # Get all keys
values = person.values()   # Get all values
```

Control Flow: If Statements

Basic Syntax:

```
age = 20

if age < 13:
    print("Child")
elif age < 20:
    print("Teenager")
else:
    print("Adult")
```

Comparison Operators:

```
x = 5
y = 10

print(x == y)    # Equal to
print(x != y)    # Not equal to
print(x < y)     # Less than
print(x <= y)    # Less than or equal
print(x > y)     # Greater than
print(x >= y)    # Greater than or equal
```

Control Flow: Loops

For Loops:

```
# Iterating over a list
for num in [1, 2, 3]:
    print(num)

# Using range
for i in range(3):    # 0, 1, 2
    print(i)

# Loop with enumerate
for i, val in enumerate(["a", "b", "c"]):
    print(f"{i}: {val}")
```

While Loops:

```
# Continue until condition is met
count = 0
while count < 3:
    print(count)
    count += 1

# Break and continue
for i in range(5):
    if i == 2:
        continue    # Skip 2
    if i == 4:
        break        # Stop at 4
    print(i)
```

Functions

Function Definition:

```
def greet(name):  
    """Greet a person by name."""  
    return f"Hello, {name}!"  
  
# Calling the function  
message = greet("Alice")  
print(message)    # Hello, Alice!
```

Function Parameters:

```
# Default parameters  
def power(x, exponent=2):  
    return x ** exponent  
  
print(power(2))      # 4 (22)  
print(power(2, 3))   # 8 (23)  
  
# Multiple parameters  
def calc_stats(numbers):  
    return sum(numbers), len(numbers)  
  
total, count = calc_stats([1,2,3,4])
```

Using AI

AI in the Job Market

- Interview Expectations Vary
 - Some employers assess AI-assisted development
 - Others assess coding without AI assistance
- You Need to Prepare for Both
 - Check the rules for each interview or assessment
 - The good news: You can optimize for both simultaneously
- This Course Will Help You Do Both
 - Build deep foundational knowledge (for traditional interviews)
 - Master AI-augmented development (for AI-forward companies)

The Core Principle

- Words Have Power
 - Clear instructions help; verify the result
 - Describe the problem, constraints, and success criteria
- Domain Knowledge Improves Your Prompts
 - Better vocabulary → better prompts → better results
 - Domain expertise lets you ask questions others can't even formulate
- How Do You Build This Mastery?
 - Deep domain knowledge and hands-on experience
 - There's no shortcut — you have to build things

AI-Maxing: Our Philosophy

- Use AI to learn; follow each assignment's rules
- But here's the paradox:
 - Those who take AI the furthest are those who build intimate knowledge
 - Knowing the concepts helps you catch AI mistakes
- The winning strategy:
 - Practice both with AI and without it
 - But intentionally build deep understanding of the course material
 - Use AI as your teacher, not just your coder

Use AI as Your Teacher

- Maximize the questions you ask
 - Ask it to test your thinking
 - Ask it to explain why, not just how
 - Ask it to challenge your assumptions
- Exploit your unprecedented access to knowledge
 - The rate at which you can explore rabbit holes has never been greater
 - Dig into the underlying mechanisms of everything you interact with
- Give AI permission to ask YOU questions
 - This clarifies your own thinking
 - “AI is most unhelpful when you don’t know how to ask well”

Introducing Claude Code

What is Claude Code?

- An agentic coding tool that runs in your terminal
 - Claude can read, write, and edit files directly in your project
 - Can explore project files; provide relevant context
 - Can run commands, tests, and debug issues
- Why use a coding agent?
 - Direct access to your development environment
 - No copy-pasting code back and forth
 - Seamless integration with your workflow
- Perfect for this course
 - Use it to explore, debug, and learn
 - Let it help you understand complex code

Getting Started with Claude Code

- **Prerequisite:** Paid Claude plan or Console/API access

- **Install (Mac/Linux/WSL):**

```
curl -fsSL https://claude.ai/install.sh | bash
```

- **Start using Claude Code:**

```
cd your-project
```

```
claude
```

- You'll be prompted to log in on first use
- Windows PowerShell: irm https://claude.ai/install.ps1 | iex

- **Docs:** code.claude.com/docs/en/setup

Claude Code Demo

Our Recommendation

- Our demo: Claude Code (Anthropic)
 - Read files, make changes, and run checks
 - Review diffs and understand generated code
 - Use current tool/model options in your account
- Other options to explore:
 - Claude, ChatGPT, Gemini — chat-based tutoring
 - Codex (OpenAI) — coding agent; Cursor — AI editor
- Pro Tip: Use multiple tools
 - Compare tools on the same small coding task
 - Check correctness, cost, and usage limits

Practical Tips

- Always give AI context — it can't read your mind
 - What you're working on, what you need help with
 - The error message (full stack trace)
 - The relevant code
- Ask AI to ask YOU questions
 - It will fish for the missing context it needs
- Treat AI like a tutor, not a code generator
 - Ask it to explain and comment its code
 - Tell it your learning goals
- Ask follow-ups; usage limits depend on your plan

HW #1

HW #1 - Getting Your Feet Wet

- Due Monday, October 19, 2026
- See the course repo and Canvas for HW1 instructions
- What it covers:
 - Setting up your Python environment
 - Basic Python coding
 - Using Git
 - Setting up Claude Code